

Urovo K388 Print Development Documentation

[V1.1]

Contents

Urovo K388 Print Development Documentation.....	1
[V1.1].....	1
Overview	3
Key features	3
System Requirements	3
Quick Start	3
1. Add dependencies.....	3
2. Basic Usage Procedure.....	4
3. Core Class Description	4
UPrinterManager – Main Entry Class.....	4
Method Description.....	5
Connection management methods.....	5
Status monitoring method.....	5
Basic Printing Control Methods.....	6
RFID Operation Method.....	7
Print task statement generation method.....	8
Page setup method.....	9
Graphic drawing methods.....	9
Text printing methods.....	10
Barcode printing method.....	11
Image printing method.....	12
Printing control methods.....	12
Advanced printing features.....	14
Tools and methods.....	16
Bluetooth printing.....	16
Printer Setting.....	17
Usage example.....	18
Basic printing process.....	18
RFID Operation Example.....	18
Advanced printing features.....	19
Bluetooth printing.....	19
Printer setting.....	20
Precautions	20
Version information	20
Error handling	21
Exception types.....	21
Status Code Explanation.....	22

Overview

The Urovo K388 printer SDK is a printer driver development kit designed specifically for the Android platform. It provides a complete interface for printing functions and supports text, barcode, QR code, image printing, and RFID operation.

Key features

- Supports printing text, barcodes, QR codes, and images.
- Flexible print configuration and style settings
- Real-time printer status monitoring
- Complete error handling mechanism
- Thread-safe API design
- Supports batch print job management

System Requirements

- Android API level 19 (Android 4.4) and above
- K388 series printers

Quick Start

1. Add dependencies

Add the SDK JAR package to the project's `libs` directory and configure it in `build.gradle`:

```
gradle
dependencies {
implementation files('libs/UK388PrintLibrary_v1.0.25_1011.jar')
}
```

2. Basic Usage Procedure

```
UPrinterManager printerManager = UPrinterManager.getInstance(context);
try {
    // Connect printer
    printerManager.connect();
    // Clear the printer buffer
    printerManager.flushBuffer();
    // Check status
    if (printerManager.getPrinterStatus().isReady()) {
        // Settings page
        printerManager.pageSetup(560, 400);
        // Print text
        printerManager.drawText(50, 50, "Hello World", 16, 0, 0, false, false);
        // Print barcode
        printerManager.drawBarCode(50, 100, "123456", 1, 0, 2, 50);
        // Execute print
        String string = printerManager.print(0, 1);
        printerManager.printByText(string);
    }
} catch (PrinterException e) {
    e.printStackTrace();
} finally {
    // Disconnect printer
    printerManager.disconnect();
}
```

3. Core Class Description

UPrinterManager – Main Entry Class

The main entry point for printer functions, responsible for device connection, status management, and print job execution.

Method Description

Connection management methods

```
/**  
Connect printer  
* @throws PrinterException Throws an exception when the connection fails.  
/  
public void connect() throws PrinterException
```

```
/**  
Disconnect the printer.  
/  
public void disconnect()
```

```
/**  
* Check if the printer is connected.  
@return true indicates that the connection is established.  
/  
public boolean isConnected()
```

```
/**  
Clear printer buffer  
/  
public void flushBuffer()
```

Status monitoring method

```
/**  
Get printer status  
* @return Printer status enumeration  
/  
public PrinterStatus getPrinterStatus()
```

```
/**  
* Get printer status description  
* @return status description text  
/  
public String getPrinterStatusDescription()
```

```

/**
 * Set up printer status listener
 * @param listener Status listener
 /
public void setPrinterStatusListener(PrinterStatusListener listener)

```

Basic Printing Control Methods

```

/**
 * Send raw data directly to the printer to execute printing.
 * @param data The data to be sent.
 @return true indicates successful transmission.
 /
public boolean printByData (byte[] data)

```

```

/**
 * Send text commands directly to the printer to execute printing (using GBK encoding)
 * @param text The text to send
 @return true indicates successful transmission.
 /
public boolean sendText(String text)

```

```

/**
 * Set print density
 * @param level Concentration level (0-3)
 /
public void setContrast(int level)

```

```

/**
 * Set print speed
 * @param level Speed level (0-5)
 /
public void setSpeed(int level)

```

```

/**
 * Set bold mode
 * @param level 0 = disable bold, >0 = enable bold
 /
public void setBold(int level)

```

```

/**

```

```
* Delayed printing
* @param time Delay time (unit: 1/8 second)
/
public void setPrintWait(int time)
```

RFID Operation Method

```
/**
 * RFID write operation
 * @param bank Storage area type (0: Reserved area, 1: EPC area, 2: TID area, 3: User area)
 * @param password Access password
 * @param offset Offset address (must be an even number, minimum 4)
 * @param data The data to be written.
 * @param len Data length (must be an even number)
 @return true indicates successful write.
 /
public boolean rfidWrite(int bank, int password, int offset, byte[] data, int len)
```

```
/**
 * RFID reading operation
 * @param bank Storage area type (0: Reserved area, 1: EPC area, 2: TID area, 3: User area)
 * @param password Access password
 * @param offset Offset address (must be an even number, minimum 4)
 * @param data A buffer used to store read data.
 * @param len The length of the data to be read (must be an even number)
 @return true indicates successful read.
 /
public boolean rfidRead(int bank, int password, int offset, byte[] data, int len)
```

```
/**
 * Set RFID read power
 * @param power power
 */
public boolean setReadPower(int power )
```

```
/**
 * Set RFID write power
 * @param power power
 */
public boolean setWritePower(int power )
```

```
/**
 * Obtain RFID read power
 * @return int read power
```

```

/*
public int getReadPower()
/**
 * Obtain RFID write power
 * @return int write power
 */
public int getWritePower()
/**
 * Obtain RFID tag power
 * @return int tag power
 */
public int getCurLabelPower()
/**
 * Get RFID reader status
 * @return true indicates the card reader is enabled ; false indicates it is disabled .
 */
public boolean getRfidCardState()

```

Print task statement generation method

```

/**
Perform a print operation
 * @param horizontal Horizontal parameter
 * @param skip Paper feed mode (0: no paper feed; 1: label detection; 2: left black label
detection; 3: right black label detection)
 * @return Print instruction string
 /
public String print(int horizontal, int skip)
/**
Performs a print operation (returns a byte array).
 * @param horizontal Horizontal parameter
 * @param skip Paper feed method
 * @return Print instruction byte array
 /
public byte[] printByte(int horizontal, int skip)
/**
Perform a print operation (using default parameters).
 * @return Print instruction string
 /
public String print()

```



```

/**
 * Set the number of prints
 * @param time Number of times to print, defaults to 1 time
 /
public void setPrintTime(int time)

```

Page setup method

```

/**
 * Set page width and height
 * @param pageWidth Paper width (dots)
 * @param pageHeight Paper height (in dots)
 /
public void pageSetup(int pageWidth, int pageHeight)

```

```

/**
 * Set page width and height (with rotation and paper feed options)
 * @param pageWidth Paper width
 * @param pageHeight Paper height
 * @param r Rotation angle (0: no rotation; 1: rotate 90 ° ; 2: rotate 180 ° ; 3: rotate 270 ° )
 * @param gap Paper feed mode (0: no paper feed; 1: label detection; 2: left black label detection; 3: right black label detection)
 /
public void pageSetup(int pageWidth, int pageHeight, int r, int gap)

```

Graphic drawing methods

```

Basic graphic drawing
/**
Print rectangle
 * @param lineWidth: Border line width
 * @param top_left_x The x-coordinate of the top-left corner of the rectangle.
 * @param top_left_y The y-coordinate of the top-left corner of the rectangle.
 * @param bottom_right_x The x-coordinate of the bottom right corner of the rectangle.
 * @param bottom_right_y The y-coordinate of the bottom right corner of the rectangle.
 /
public void drawBox(int lineWidth, int top_left_x, int top_left_y, int bottom_right_x, int

```

bottom_right_y)

```
/**
 * Print solid lines
 * @param lineWidth Line width
 * @param start_x The x-coordinate of the starting point of the line.
 * @param start_y The y-coordinate of the starting point of the line.
 * @param end_x The x-coordinate of the end point of the line.
 * @param end_y The y-coordinate of the end point of the line.
 * @param fullline Whether to use a full line
 /
public void drawLine(int lineWidth, int start_x, int start_y, int end_x, int end_y, boolean fullline)
```

```
/**
 * Reverse white line segment
 * @param x0 Starting point x-coordinate
 * @param y0 Starting point y-coordinate
 * @param x1 End point x-coordinate
 * @param y1 End point y coordinate
 * @param width Line segment width
 /
public void drawINVERSE(int x0, int y0, int x1, int y1, int width)
```

Text printing methods

```
/**
 * Printed text (simplified version)
 * @param text_x Starting x-coordinate of the text
 * @param text_y Starting y-coordinate of the text
 * @param text The text content to be printed.
 * @param fontSize Font size
 * @param rotate Rotation angle (0, 90, 180, 270)
 * @param bold Print line thickness
 * @param reverse Print in reverse
 * @param underline Underscore
 /
public void drawText(int text_x, int text_y, String text, int fontSize, int rotate, int bold, boolean reverse, boolean underline)
```

```
/**
Print text (specify font type - int)
 * @param text_x Starting x-coordinate of the text
 * @param text_y Starting y-coordinate of the text
```

```

* @param text The text content to be printed.
* @param fontType Font type (int)
* @param fontSize Font size
* @param rotate Rotation angle
* @param bold Print line thickness
* @param reverse Print in reverse
* @param underline Underscore
/

public void drawText(int text_x, int text_y, String text, int fontType, int fontSize, int rotate, int
bold, boolean reverse, boolean underline)

/**
Print text (specify font type - string)
* @param text_x Starting x-coordinate of the text
* @param text_y Starting y-coordinate of the text
* @param text The text content to be printed.
* @param fontType Font type (string)
* @param fontSize Font size
* @param rotate Rotation angle
* @param bold Print line thickness
* @param reverse Print in reverse
* @param underline Underscore
/

public void drawText(int text_x, int text_y, String text, String fontType, int fontSize, int rotate, int
bold, boolean reverse, boolean underline)

```

Barcode printing method

```

/**
* Print barcodes
* @param start_x Starting x-coordinate of the barcode
* @param start_y Starting y-coordinate of the barcode
* @param text The content of the barcode.
* @param type Barcode type (0:CODE39; 1:CODE128; 2:CODE93; 3:CODEBAR; 4:EAN8; 5:EAN13;
6:UPCA; 7:UPC-E; 8:I2OF5)
* @param rotate Rotation angle (0: horizontal barcode; 1: vertical barcode)
* @param linewidth Barcode line width
* @param height Barcode height
/

public void drawBarCode(int start_x, int start_y, String text, int type, int rotate, int linewidth, int
height)

```

```

/**
 * Print QR code
 * @param start_x Starting x-coordinate of the QR code
 * @param start_y Starting y-coordinate of the QR code
 * @param text QR code content
 * @param rotate Rotation angle
 * @param ver QR code magnification factor (1-32, default 6)
 * @param lel QR code version number (1-40, 0 indicates automatic)
 /
public void drawQrCode(int start_x, int start_y, String text, int rotate, int ver, int lel)

```

Image printing method

```

/**
 * Print image
 * @param start_x X coordinate of the start of the bitmap
 * @param start_y Y coordinate of the start of the bitmap
 * @param bmp bitmap
 /
public void drawGraphic(int start_x, int start_y, Bitmap bmp)

/**
 * Print images from files
 * @param start_x The X coordinate of the starting point of the image.
 * @param start_y The Y coordinate of the starting point of the image.
 * @param filepath Image file path
 /
public void drawImageFormFile(int start_x, int start_y, String filepath)

```

Printing control methods

```

/**
 Paper feed before printing
 * @param len Paper feed distance in dots (8 dots = 1mm)
 /
public void prefeed(int len)

```

```
/**
 * Paper feed after printing
 * @param len Paper feed distance in dots (8 dots = 1mm)
 /
public void postfeed(int len)
```

```
/**
 * Left alignment
 /
public void alignLeft()
```

```
/**
 * Right alignment
 /
public void alignRight()
```

```
/**
Center Alignment
 /
public void alignCenter()
```

```
/**
 * End command
 /
public void end()
```

```
/**
 * Ignore instructions
 /
public void about()
```

```
/**
 * Numbers automatically increase or decrease
 * @param mun The increment or decrement value (any integer within  $\pm 65535$ )
 /
public void count(int mun)
```

```
/**
 * Character magnification
 * @param w Width magnification factor
 * @param h Magnification factor
 /
public void setMag(int w, int h)
```

```
/**
 * Set character spacing
 * @param spacing Character spacing (spacing*0.125mm)
 /
public void setSP(int spacing)
```

```
/**
 * Set up batch printing (press the paper feed button to print the next sheet)
 /
public void setPace()
```

```
/**
 * Adjust contrast
 * @param level Contrast level (0=default, 1=medium, 2=black, 3=very black)
 /
public void contRast(int level)
```

```
/**
 * Set print speed
 * @param level Speed level (0-5, 0 is the slowest speed)
 /
public void speed(int level)
```

```
/**
 * Set delayed printing
 * @param time Delay time (unit: 1/8 second)
 /
public void printWait(int time)
```

Advanced printing features

```
/**
 * Set barcode identifier
 * @param font Identifier font number
 * @param size Recognition character size
 * @param offset The offset number of dots from the barcode (8 dots = 1mm)
 * @param rotate Rotation angle (0: horizontal barcode; 1: vertical barcode)
 * @param width The width of the narrow barcode (number of dots)
 * @param ratio The ratio of wide barcodes to narrow barcodes
 * @param height Barcode height (number of dots, 8 dots/mm)
 * @param x The X-axis coordinate of the barcode's start point
 * @param y The Y-axis coordinate of the barcode's starting point
```

```
* @param data Barcode data
/  
public void barcodeText(int font, int size, int offset, int rotate, int width, int ratio, int height, int x,  
int y, String data)
```

```
/**  
* Text-linked printing  
* @param x Start coordinate of X-axis  
* @param y: Y-axis start coordinate  
* @param concat List of associated text strings  
* @param rotate Rotation angle  
/  
public void textConcatenation(int x, int y, List<_PrinterPageImpl.CONCAT> concat, int rotate)
```

```
/**  
Multi-line printing  
* @param height Height of each line of text  
* @param fontType Font type  
* @param fontSize Font size  
* @param x Start coordinate of X-axis  
* @param y: Y-axis start coordinate  
* @param rotate Rotation angle  
* @param strs The array of text to be printed  
/  
public void multiLine(int height, int fontType, int fontSize, int x, int y, int rotate, String... strs)
```

```
/**  
* Set the grayscale level of the watermark text  
* @param level The grayscale value of the watermark text (0-255, the larger the value, the darker  
the color).  
/  
public void backGround(int level)
```

```
/**  
* Print watermark text  
* @param font Font name or font ID  
* @param size Font size  
* @param x Start coordinate of X-axis  
* @param y: Y-axis start coordinate  
* @param lev Watermark depth (0-255)  
* @param str The text to be printed  
* @param rotate Rotation angle  
/  
public void bkText(int font, int size, int x, int y, int lev, String str, int rotate)
```

--

Tools and methods

```
/**
 * Retrieve the print command string (for debugging)
 * @return Print instruction string
 /
public String getPrintPrintStr()
```

```
/**
 * Get SDK version information
 * @return SDK version string
 /
public String getSDKVersion()
```

```
/**
 * Get printer version information
 * @return Version information string
 /
public String version()
```

```
/**
Release resources
 /
public void release()
```

Bluetooth printing

```
/**
 * Configure a remote service listener
 * @param listener
 /
public void setRemoteServiceListener(RemoteServiceListener listener) {
this.remoteServiceListener = listener;
}
```

```
**
 * Set print path
 * @param route
 /
public void setPrintRoute(PrintRoute route)
```


Printer Setting

```
/**
 * Get Printer Firmware version
 * @return printer firmware version
 * /
public String getFirmWareVersion()
```

```
/**
 * Printer firmware upgrade
 * @param firmwareFilePath the absolute path of firmware file (.upd)
 * @throws PrinterException wrong params,abnormal status,write failed,read/write file failed
 */
public synchronized void upgradeFirmware(String firmwareFilePath) throws PrinterException
```

```
/**
 * Rewind print setting
 * @param enable if set rewind print mode
 * @return set result true:successful false:failed
 */
Public synchronized boolean setRewindPrint(boolean enable)
```

```
/**
 * Multi-print setting
 * @param delay time(ms)example:0(means disable),10,100
 * @return set result true:successful false:failed
 */
public synchronized boolean setMultiPrintDelay(int delayMs)
```

```
/**
 * Print mode setting
 * @param 0=normal, 1=high speed, 2=low speed
 * @return set result true:successful false:failed
 */
public synchronized boolean setPrintMode(int mode) throws PrinterException
```

```
/**
 * Print darkness setting
 * @param darkness From -5 to +5
 * @return set result true:successful false:failed
 */
public synchronized boolean setPrintDarkness(int darkness) throws PrinterException
```

Usage example

Basic printing process

```
UPrinterManager printerManager = UPrinterManager.getInstance(context);
try {
    // Connect printer
    printerManager.connect();
    // Clear the printer buffer
    printerManager.flushBuffer();
    // Check status
    if (printerManager.getPrinterStatus().isReady()) {
        // Settings page
        printerManager.pageSetup(560, 400);
        // Print text
        printerManager.drawText(50, 50, "Hello World", 16, 0, 0, false, false);
        // Print barcode
        printerManager.drawBarCode(50, 100, "123456", 1, 0, 2, 50);
        // Execute print
        String string = printerManager.print(0, 1);
        printerManager.printByText(string);
    }
} catch (PrinterException e) {
    e.printStackTrace();
} finally {
    // Disconnect printer
    printerManager.disconnect();
}
```

RFID Operation Example

```
// RFID writing
byte[] data = "RFID Test Data".getBytes();
boolean writeSuccess = printerManager.rfidWrite(3, 0, 4, data, data.length);
// RFID reading
byte[] readBuffer = new byte[32];
boolean readSuccess = printerManager.rfidRead(3, 0, 4, readBuffer, readBuffer.length);
if (readSuccess) {
    String readData = new String(readBuffer).trim();
}
```

```
Log.d(TAG, "RFID data read: " + readData);
```

Advanced printing features

```
// Set printing parameters
printerManager.setContrast(2); // Print density
printerManager.setSpeed(3); // Printing speed
printerManager.setBold(1); // Bold mode
// Multi-line printing
printerManager.multLine(30, 0, 16, 50, 150, 0, "First line", "Second line", "Third line");
// Watermark printing
printerManager.bkText(0, 16, 100, 200, 128, "Watermark Text", 0);
```

Bluetooth printing

```
if (uPrinterManager == null) {
    uPrinterManager = UPrinterManager.getInstance(mContext);
    if (uPrinterManager == null) {
        binding.tvRes.setText("AIDL Remote Service Test res:false, UPrinterManager not initialized");
        return;
    }
}

uPrinterManager.setRemoteServiceListener(new UPrinterManager.RemoteServiceListener() {
    @Override
    public void onConnectionChanged(String deviceAddr, String deviceName, boolean connected) {
        Log.d(TAG, "Remote AIDL onConnectionChanged addr=" + deviceAddr + ", name=" + deviceName
            + ", connected=" + connected);
    }

    @Override
    public void onScanData(String deviceAddr, String deviceName, String scanData) {
        Log.d(TAG, "Remote AIDL onScanData data=" + scanData);
        uPrinterManager.pageSetup(200, 100);
        uPrinterManager.drawText(0,0,"Test",4,4,0,false,false);
        String text = uPrinterManager.print(0,1);
        Log.e(TAG, "onScanData text:"+ text);
        uPrinterManager.printByText(text);
    }
});
```

```

}

@Override
public void onPrinterStatus(int status) {
    Log.d(TAG, "Remote AIDL onPrinterStatus=" + status);
}
});

uPrinterManager.setPrintRoute(UPrinterManager.PrintRoute.REMOTE_ONLY);
int status = uPrinterManager.getPrinterStatus().getCode();

```

Printer setting

```

public void getFirmwareVersion() {
    String versionName = uPrinterManager.getFirmWareVersion();
    Log.d(TAG, "Firmware Version:" + versionName);
}

public void upgradeFirmware(Uri firmwareUri) {
    uPrinterManager.upgradeFirmware(firmwareUri);
}

public void setPrinter() {
    uPrinterManager.setRewindPrint(true); //Set rewind print
    uPrinterManager.setMultiPrintDelay(100); //Set 100ms Delay time
    uPrinterManager.setPrintMode(1); //Set high speed print mode
    uPrinterManager.setPrintDarkness(3); //Set +3 print darkness
}

```

Precautions

1. ****Coordinate Units****: All coordinate parameters are in points (dots), 8 dots = 1 mm.
2. ****RFID Operation****: The offset address must be an even number and not less than 4; the data length must be an even number.
3. ****Encoding Format****: Text printing uses GBK encoding.
4. ****Resource Management****: Always call the `release()` method to release resources after use.
5. ****Exception Handling****: All operations that may fail should be handled with exception handling.

Version information

- SDK Version: Urovo K388 Printer SDK v2.3.0
- Compatibility: Android API level 19 and above
- Supported printers: Urovo K388 series

This document provides complete method descriptions for the UPrinterManager class to help developers quickly get started using the printer SDK.

Error handling

Exception types

Exception Class	illustrate	Handling suggestions
PrinterException	Common printer errors	Check connection and status

IllegalArgumentException	Parameter error	Check configuration parameters
--------------------------	-----------------	--------------------------------

Status Code Explanation

| Status Code | Status | Description |

status codes	state	illustrate
0	OK	Printer is working properly
1	COVER_OPEN	Paper tray lid open
2	NO_PAPER	Paper shortage
3	PAPER_ERROR	Paper error
4	UNKNOWN_ERROR	Unknown error
5	DISCONNECTED	Not connected